

S4.3 — Demo Rehearsal Preparation & Fallback Plan

Project: AI ServiceNow Support Assistant — BARQ G4

Sprint: 4 of 4 — Demo Day Readiness

Owner: Omar Reda

1. Rehearsal Preparation

1.1 What the demo must show (the “happy path”)

Framing note: this presentation is business-first, not a technical deep dive. The audience cares about the problem being solved, the trust boundary (the system suggests, a human decides), and the measurable result — not webhook contracts or Celery internals. Keep the technical steps below as the backbone of *what gets shown on screen*, but narrate them in plain outcome language. Save the engineering vocabulary for Q&A, where a technical question actually invites it.

The deck’s own Live Demo slide (slide 8) already defines the three on-screen moments to hit, in this exact order — rehearsal should walk through these same three moments every time, so the video and the live narration never drift apart:

1. **The ticket is picked up automatically** — no one has to notice it.
2. **The system searches the knowledge base and drafts a cited answer** — or, on a deliberately unanswerable ticket, escalates instead (see Scenario 5 in the Fallback Plan — this path is not improvised, it’s already built and tested).
3. **The suggestion is ready for a human** — approve, edit, or reject, right on the ticket.

Underneath those three moments, the real sequence actually running on screen is:

1. A support ticket comes in (an incident is created/updated in ServiceNow).
2. The system picks it up automatically.
3. It searches the team’s own knowledge base for a relevant past answer.
4. It either drafts a suggested response (grounded in a real knowledge article, cited) or flags the ticket for a human to handle.
5. The suggestion appears directly in the ticket.
6. Every decision is fully traceable after the fact in Langfuse.

(The technical names behind each beat — the webhook receiver, Celery queue, the Qdrant search, Langfuse trace — are reference detail for the presenter’s own preparation and for Q&A, not things to say out loud during the main narration.)

A note on the deck itself: S4.1 (Sherouk Mohamed) and S4.2 (Menna Ahmed Abd Elkader) jointly own the actual final presentation deck. The `BARQ_G4_Demo_Day.pptx` built alongside this plan exists so the time budget and narration below could be rehearsed against real,

concrete slide content rather than an abstract outline — it’s offered as a starting point or reference, not a substitute for their deliverable. Sync with them before Demo Day on whether the team presents from this file, their own, or a merged version — and if the final deck’s slide count or order differs, the time budget in 1.2 needs re-checking against whatever is actually used, since it’s built around this specific 12-slide structure.

1.2 Time budget — 10 minutes total, video included

The 10-minute limit covers **everything**: all 12 slides, the spoken narration, and the embedded demo video. The video is not extra time on top — it’s slide 8 of the deck. This is the deck’s own official per-slide timing (from the presentation template), not an approximation — rehearse against these exact numbers.

#	Slide	Target time
1	Title	0:15
2	Team	0:30
3	Problem	1:00
4	Solution	1:00
5	Goals & users	0:45
6	How it works	1:15
7	Tools & tech	0:45
8	Live demo (video)	2:00
9	Results	1:00
10	Challenges & lessons	0:45
11	Next steps	0:30
12	Close & Q&A	0:15
	Total	10:00

Narration stays business-first throughout (per 1.1’s framing note) — the slide titles above are internal rehearsal shorthand, not a script to read aloud literally.

Video-trimming rule: if the live-demo footage runs long in rehearsal, cut dead moments out of the video itself (container startup, waiting on an API call) rather than eating into the 2:00 allocation or any other slide’s time — keep every moment where something visibly changes (the ServiceNow form updating, the Langfuse trace appearing). A tight 2-minute video showing the real sequence beats a rushed 3-minute one eating into Results or Next Steps.

This is only real once it’s been timed against an actual clock with an actual person talking — the rehearsal schedule in 1.4 exists specifically to catch a plan that reads fine on paper but runs long out loud.

1.3 Pre-rehearsal checklist (must be true before any rehearsal run starts)

- ☐ ServiceNow PDI reachable, correct AI_FIELD_PREFIX confirmed for the demo instance (scope prefixes differ per PDI — confirm the exact value before the first rehearsal, don't assume a prior value still applies)
- ☐ Qdrant collection populated with the real knowledge base articles the demo incident is expected to match against
- ☐ Docker stack (api, worker, redis, qdrant) brought up fresh and confirmed healthy (docker compose ps, curl /health) at least 15 minutes before rehearsal starts — not assumed still running from a previous session
- ☐ A real LiteLLM/Gemini key confirmed valid with a throwaway test call, same day as rehearsal (proxy keys and quotas have been unreliable during development — confirm fresh each time, don't trust yesterday's test)
- ☐ At least 2 reserved, untouched real incidents identified in the PDI — one expected to produce a confident answer, one expected to escalate — so the demo shows both terminal paths, not just one
- ☐ Vision/embedding response caching warmed (where applicable) so the recorded run doesn't stall on a slow API call
- ☐ A clean Langfuse trace from a successful rehearsal run saved/bookmarked, to use as backup evidence if needed (see Section 2)

1.4 How the rehearsal runs

1. **Technical dry run (no audience, operator only).** Run the real sequence from 1.1 against a reserved real incident, twice, back to back — once ending in a confident suggestion, once ending in an escalation, so both of the deck's two possible outcomes are rehearsed, not just one. Record the Arcade capture during the second run once the first confirms nothing's broken.
2. **Timed read-through.** Presenter(s) narrate against the time budget in 1.2, with an actual stopwatch, segment by segment. If any segment runs over, cut content from that segment specifically rather than borrowing time from another — the budget has no slack built in by design.
3. **Final rehearsal, day before Demo Day.** One last full run, specifically re-validating the pre-rehearsal checklist above (credentials and PDI state can silently drift day to day — confirmed repeatedly during development).

2. Fallback Plan

Every scenario below is grounded in a failure **actually observed and reproduced** during real testing of this system — not hypothetical. **Scenario 1 specifically is not new material written for this task** — it's the exact failure-handling pattern S3.4 (Agent Tools & Autonomous Execution) was already required to build and demonstrate: *"clean error logging, incident left in recoverable in_progress status with a failure note, and worker continuity."* That requirement was met and evidenced directly — real ServiceNow, real

Qdrant, a deliberately invalid input, and a healthy job processed immediately afterward in the same run. This section reuses that proof rather than inventing a new untested scenario.

Scenario 1 — An unexpected error occurs mid-processing (already built and proven in S3.4)

What happens: Something fails partway through a run — a ServiceNow call errors out, the data is malformed, or any other unexpected exception occurs after the incident has already been claimed.

Recovery (non-video — the system’s own error containment, already tested): This is not a recovery the presenter improvises — it’s architecture. The worker catches the failure, logs it with a full traceback, writes a failure work note back to the incident (retried once if the first write itself fails), and deliberately leaves `ai_status` and `ai_processed` untouched — so the incident stays exactly where the eligibility webhook already claimed it, recoverable, not corrupted. The worker process itself never crashes: it moves on and processes the next job normally. **This exact sequence was directly tested and evidenced during S3.4** — a deliberately invalid input was run, followed immediately by a healthy job in the same process, proving continuity, not just isolated error-handling.

What to say on stage, if this comes up (live or in Q&A): *“If something fails mid-run, the system doesn’t lose the ticket or crash — it leaves it exactly where it was claimed, logs what happened, and keeps working on the next one. We built this deliberately and tested it as part of an earlier sprint — it’s not a hope, it’s proven behavior.”*

Scenario 2 — LLM provider returns a 503 / rate-limit error mid-demo

What happens: The Gemini model (direct or via the LiteLLM proxy) returns 503 UNAVAILABLE or a similar transient error during the agent’s reasoning step. Observed multiple times during development, at unpredictable moments.

Recovery (non-video — retry): The system already retries transient errors automatically once before failing permanently. If the live attempt still fails, the operator re-runs the exact same test event (`test_webhook.ps1`, same or fresh `event_id`) — in every observed case during development, a second or third attempt succeeded shortly after. This is a genuine retry, not a trick: budget 30–60 seconds of narration (“the model’s reasoning step, like any live AI call, occasionally needs a retry — here’s that happening now”) while the operator re-triggers it.

Fallback if retries don’t resolve it in time: Switch immediately to the pre-recorded Arcade video segment for this step.

Scenario 3 — Docker stack not running / a container has stopped

What happens: The worker or api container isn’t up, or stopped silently between rehearsal and the live slot (observed during development — closing the terminal window running `docker compose up` stops the whole stack without an obvious error).

Recovery (non-video — pre-seeded state check + restart): Run `docker compose ps` as the very first action after walking on stage, before touching anything else. If anything isn’t

Up, run `docker compose up --build` immediately — rebuilds are fast (cached layers, ~3 seconds in testing) once the images already exist locally from rehearsal. Narrate this as “let’s confirm our services are live” rather than treating it as an error — it reads as due diligence, not a mistake.

Fallback if it still won’t start in time: Switch to the pre-recorded video.

Scenario 4 — Venue network/Wi-Fi failure (can’t reach cloud services)

What happens: The venue’s network can’t reach the real ServiceNow PDI, cloud Qdrant, or the LiteLLM proxy.

Recovery (non-video — pre-seeded local fallback state): The project’s `docker-compose.yml` already supports a fully local Qdrant instance as a built-in fallback (no cloud dependency) when `QDRANT_URL` isn’t set to a cloud URL. Pre-seed this local instance with the demo’s knowledge base articles ahead of time (same seeding script used throughout development), so retrieval can run fully offline. ServiceNow itself can’t be fully replicated offline, so pair this with the cached Langfuse trace (Scenario 6) as the ServiceNow-side evidence.

Scenario 5 — The agent escalates instead of giving a confident answer

What happens: Observed consistently during development — the agent’s reasoning sometimes keeps searching rather than committing to a final answer, eventually triggering the system’s own step-budget fail-safe, which escalates to human review.

Recovery (non-video — reframe, don’t hide): This is not purely a failure case — it’s the system’s safety mechanism working exactly as designed (the task brief for the agent explicitly requires a fail-safe escalation when the agent can’t reach a confident decision). Prepare narration for both outcomes in advance: *“If the agent finds strong evidence, it suggests an answer for human review. If it doesn’t reach a confident decision within its step budget, it escalates automatically — here’s that second path.”* Rehearse with an incident reserved specifically to demonstrate each path, so neither outcome looks unplanned.

Scenario 6 — ServiceNow PDI itself is unreachable or down

What happens: The PDI instance doesn’t respond (dev instances can go into hibernation/maintenance windows unpredictably).

Recovery (non-video — cached output): Keep a saved screenshot of a completed incident’s AI Review fields (from a genuine prior run, credentials masked) and the matching Langfuse trace bookmarked and ready to open in a browser tab. Narrate it as “here’s a completed run from our testing” rather than attempting a live call that can’t succeed without PDI access.

Scenario 7 (video fallback) — Anything not covered above

Recovery (video): A full pre-recorded Arcade capture of one complete, successful run (Section 1.1), kept open and ready to play at any point, covering any failure mode not specifically anticipated above.

3. Hygiene — Credential Safety Checklist

Before any rehearsal materials, screenshots, or the final video are shared, recorded, or committed:

- ☐ No real ServiceNow password visible in any terminal window, screen recording, or screenshot
 - ☐ No real LiteLLM/Gemini API key visible anywhere (mask as sk-*** or similar in any shown .env content)
 - ☐ No real Qdrant API key visible
 - ☐ No real INCIDENT_SIGNING_SECRET / KB_SIGNING_SECRET values visible
 - ☐ ServiceNow instance URL shown is acceptable to share (it's an internal dev PDI, not a sensitive production system — confirm with the team if unsure)
 - ☐ Review the full Arcade recording once before submission specifically for any accidental credential exposure in a terminal scroll-back or an open file tab
-